Advanced Data Analytics

Emmanouil Giannakis

manosgiannakis15@gmail.com

University of Piraeus — January 23, 2025

Introduction

This project focuses on the development and evaluation of a sentiment classification system using machine learning and deep learning models. Specifically, a Support Vector Machine (SVM) classifier was trained and compared with a neural network model for text classification. Input data was transformed into vectorized representations using the Bag of Words (BoW) technique. Additionally, advanced embeddings like Word2Vec were explored to enhance text representation.

The performance of the models was rigorously evaluated based on metrics such as precision, recall, F1-score, and accuracy on a test dataset. Training time, inference time for new reviews, and overall computational efficiency were also measured and analyzed. The results provide a comparative perspective on the strengths and limitations of SVM and neural network approaches in sentiment analysis tasks.

Problem Statement

Understanding sentiment expressed in textual data is a critical aspect of natural language processing (NLP). In this project, the goal is to develop a system capable of automatically identifying the sentiment of a user based on their movie review. The sentiment can either be positive or negative, and the system must accurately classify the review into one of these categories. To achieve this, various machine learning and deep learning techniques will be employed. The challenge lies in transforming textual data into a machine-readable format while capturing the nuances of language, such as context, word order, and the overall sentiment expressed. The system must not only achieve high accuracy in classification but also be efficient in terms of training and inference time. This has real-world applications in areas like customer feedback analysis, opinion mining, and recommendation systems.

Dataset

For this project, the IMDB Review Dataset was used. This dataset is a widely utilized benchmark for sentiment analysis tasks in the field of natural language processing. It consists of 50,000 movie reviews, each labeled as either positive or negative, making it a **binary classification problem**. The dataset is evenly balanced with 25,000 positive and 25,000 negative reviews, as shown in [1] , ensuring no bias toward any particular sentiment during training. The dataset is split into two subsets:

1. **Training Set** (25,000 reviews): Used to train the machine learning and neural network models.
2. **Test Set** (25,000 reviews): Used to evaluate the performance of the trained models on unseen data.

Additionally, for this project, a validation set was created by extracting 10% of the training data. This set helps monitor the model's performance during training and fine-tune hyperparameters.

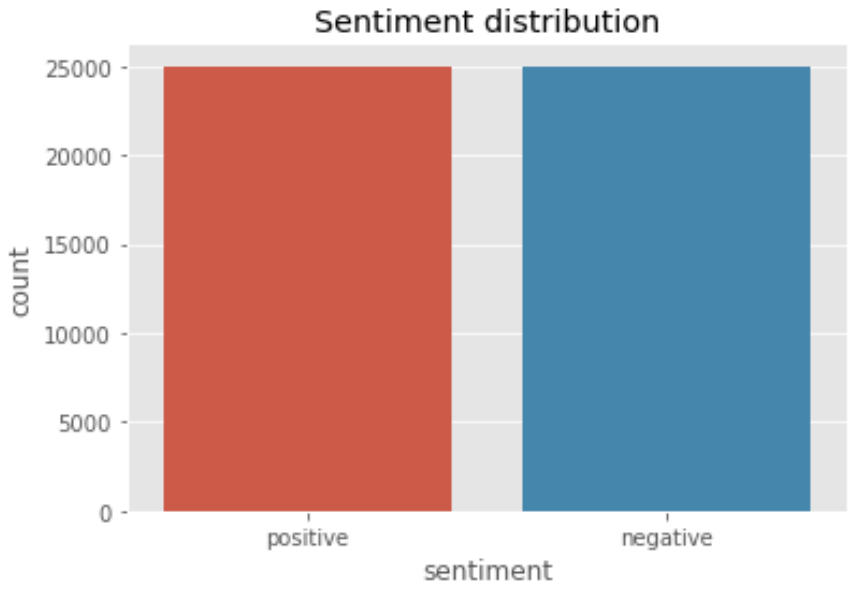


Figure 1: Balanced dataset between the two categories.

Dataset Structure Example

The dataset used for this project is structured as follows:

- **train/**
  - **pos/**: Contains positive reviews (e.g., `train/pos/1_8.txt`)
  - **neg/**: Contains negative reviews (e.g., `train/neg/0_3.txt`)
- **test/**
  - **pos/**: Contains positive reviews (e.g., `test/pos/2_10.txt`)
  - **neg/**: Contains negative reviews (e.g., `test/neg/3_4.txt`)
- **val/**
  - **pos/**: Contains positive reviews (e.g., `val/pos/5_6.txt`)
  - **neg/**: Contains negative reviews (e.g., `val/neg/6_1.txt`)

Each file contains a single movie review as plain text. Below is an example review from `train/neg/0_3.txt`:

> *Story of a man who has unnatural feelings for a pig. Starts out with an opening scene that is a terrific example of absurd comedy. A formal orchestra audience is turned into an insane, violent mob by the crazy chantings of its singers. Unfortunately, it stays absurd the WHOLE time with no general narrative eventually making it just too off-putting. Even those from the era should be turned off. The cryptic dialogue would make Shakespeare seem easy to a third grader. On a technical level, it's better than you might think with some good cinematography by future great Vilmos Zsigmond. Future stars Sally Kirkland and Frederic Forrest can be seen briefly.*

This structured organization ensures seamless processing, allowing for easy access to labeled reviews during training, validation, and testing phases.

Text Preprocessing & Vectorization

Text preprocessing and vectorization are critical steps in preparing raw text data for machine learning and deep learning models. Since these models cannot process text directly, textual data needs to be transformed into numerical representations. This section outlines the preprocessing and vectorization techniques used in this project.

1. **Text Preprocessing**: Before converting text into numerical format, preprocessing is required to clean and normalize the data.
   (a) **Tokenization**: The reviews were tokenized into individual words or n-grams using TextVectorization from TensorFlow or manually for Bag-of-Words (BoW) approaches.
   (b) **Removing Punctuation and Lowercasing**: A translation table was used **str.maketrans** to remove punctuation and converted text to lowercase.
2. **Vectorization Techniques**: Used to evaluate the performance of the trained models on unseen data.
   (a) **Bag-of-Words (BoW) Representation**: Implemented BoW with the TextVectorization layer in TensorFlow, configured to output binary vectors indicating the presence or absence of words (or n-grams) in a document.
      i. **UniGram (single words)**
      ii. **BiGram (two consecutive words)**
      iii. **TriGram (three consecutive words)**
   (b) **Word-Embeddings**:
      i. **Pre-trained Embeddings**: Leveraged GloVe embeddings by initializing the embedding layer with pre-trained weights. These weights were kept fixed during training to retain the semantic knowledge encoded.
      ii. **Trainable Embeddings**: In sequence models, word embeddings were learned during training rather than using pre-trained embeddings, allowing the model to develop task-specific representations.

Models Evaluation

0.1 N-Gram Models

Understanding the sequential patterns in text data is critical for effective text classification. *N-gram* models provide a structured approach by representing text as sequences of n-grams (consecutive tokens), capturing context at varying granularities. This study explores unigram, bigram, and trigram models, leveraging the `TextVectorization` layer in TensorFlow to preprocess and vectorize the text data. Each n-gram level offers unique advantages in identifying patterns in the text, with increasing complexity as n grows.

After analyzing a unigram, a bigram, and a trigram model, their performance was poor at the classification task. Overfitting was a persistent challenge across all three models, as indicated by the divergence between training and validation loss curves. While the unigram model provided a simple and computationally efficient baseline, it lacked contextual understanding, leading to significant misclassifications. The bigram model improved contextual recognition by capturing short-term dependencies, and the trigram model further enhanced this by identifying more nuanced patterns and phrases. However, despite these advancements, the incremental gains in accuracy and F1-score were marginal, and the models consistently failed to generalize effectively to unseen data.

This inability to mitigate overfitting rendered the performance of all three models suboptimal. High false positive and false negative rates persisted in the confusion matrices, and the precision-recall curves revealed a sharp trade-off between precision and recall, particularly in the bigram and trigram models. While adding context through n-grams improved the models' ability to recognize patterns, it also increased computational complexity and storage demands without addressing the core issue of poor generalization. This highlights the need for alternative approaches, such as regularization techniques, data augmentation, or more sophisticated architectures, to improve model robustness and classification performance.

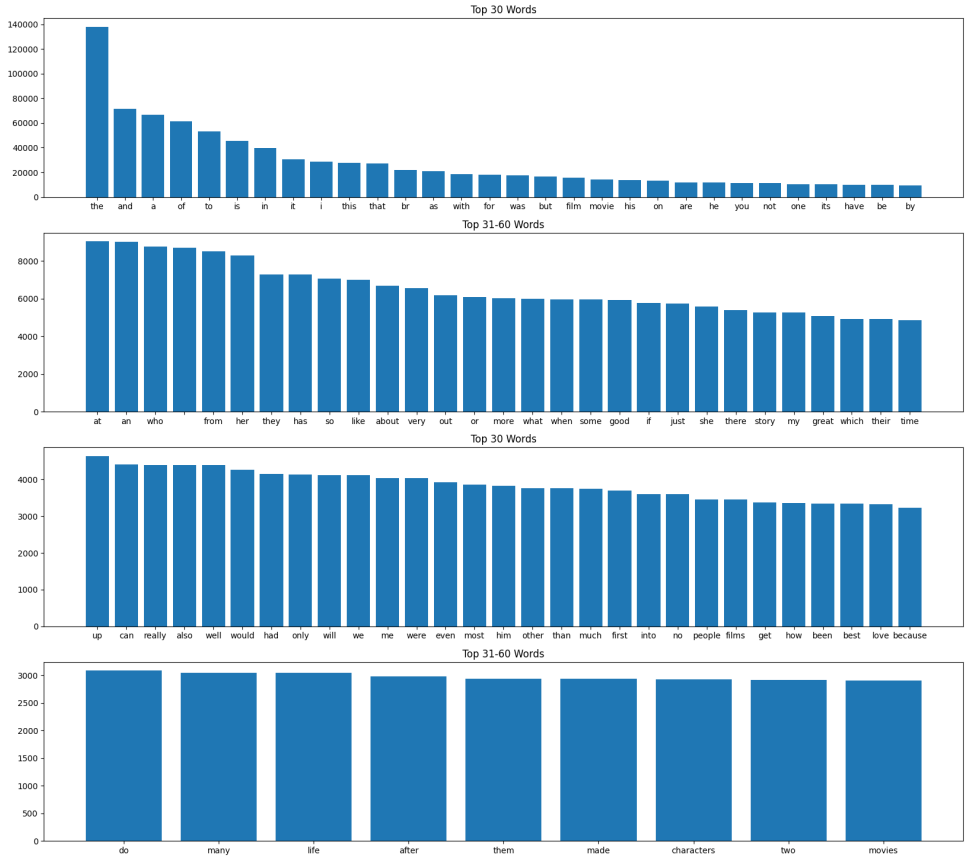
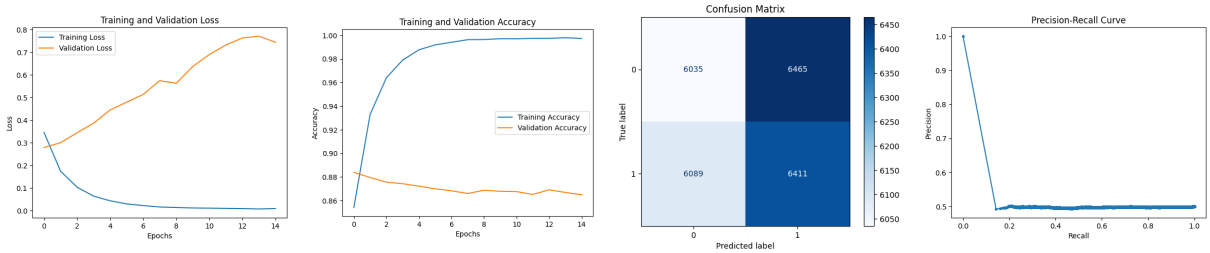


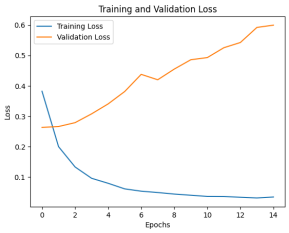
Figure 2: Most Frequent words.

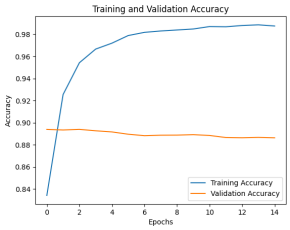
0.1.1 UniGram Model

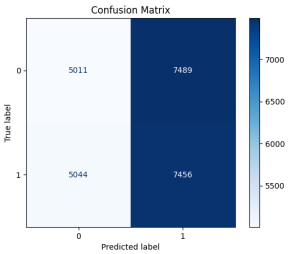


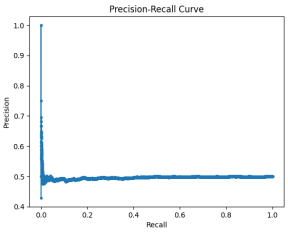
Metric	Value
Test Accuracy	0.884
Test Precision	0.876
Test Recall	0.895
Test F1-Score	0.885

0.1.2 BiGram Model



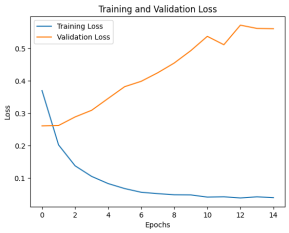


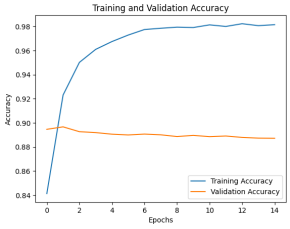


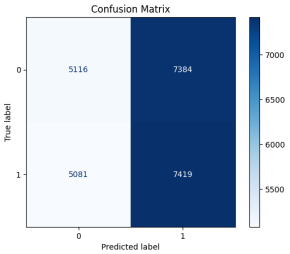


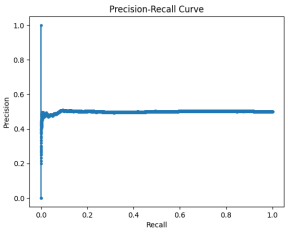
Metric	Value
Test Accuracy	0.894
Test Precision	0.909
Test Recall	0.875
Test F1-Score	0.892

0.1.3 TriGram Model







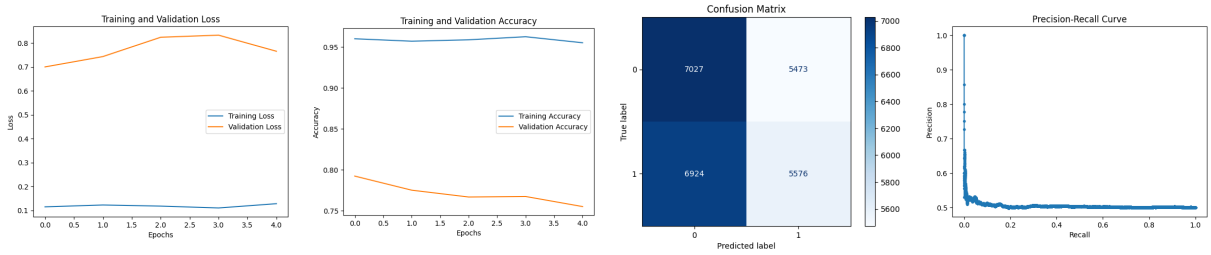


Metric	Value
Test Accuracy	0.895
Test Precision	0.894
Test Recall	0.896
Test F1-Score	0.895

0.2 Sequence-to-Sequence Utility Model

Building upon the previous text classification approaches, a utility model for Sequence-to-Sequence tasks is introduced to address the limitations of fixed-length token representations. This model leverages TensorFlow's Keras API to process sequences of variable length with integer-encoded tokens. An embedding layer converts these tokens into one-hot encoded vectors, ensuring flexibility in handling diverse vocabulary sizes.

The architecture incorporates a Bidirectional LSTM layer with 32 units, enabling the model to capture both past and future contexts in the sequence, which provides a significant advantage over simpler token-based models. A dropout layer with a 0.5 dropout rate is added to mitigate overfitting, enhancing the model's generalization capabilities. The output layer, with a single neuron and sigmoid activation, is designed for binary classification tasks.



Info: A **Bidirectional LSTM (Long Short-Term Memory)** is a type of recurrent neural network (RNN) designed to process sequence data, such as text or time-series. Its purpose is to capture dependencies in the data from both the past (previous elements in the sequence) and the future (upcoming elements). Traditional RNNs and LSTMs process sequences in one direction (either forward or backward), but bidirectional LSTMs run two LSTMs simultaneously—one in the forward direction and one in the backward direction. The outputs from both directions are combined, allowing the model to gain a more comprehensive understanding of the context. This makes bidirectional LSTMs especially useful for tasks like natural language processing, where understanding the meaning of a word often depends on both the words before and after it in a sentence.

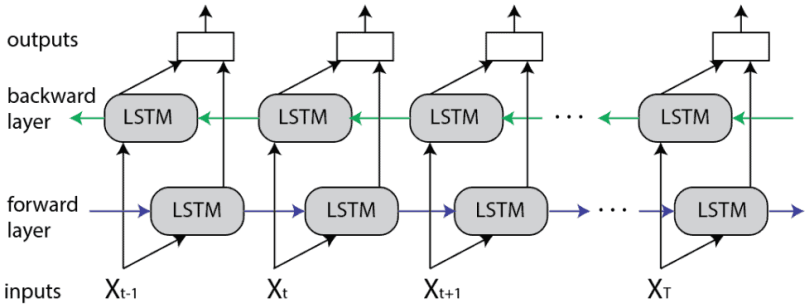


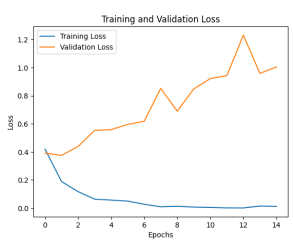
Figure 3: LSTM.

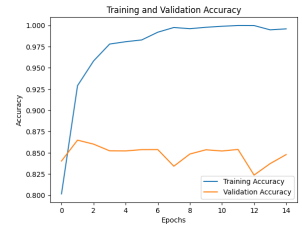
0.3 Model with Word Embeddings

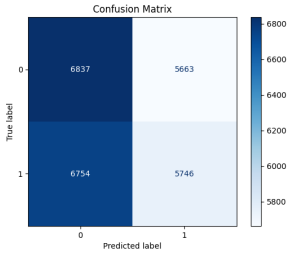
After still not achieving satisfactory results, a revised architecture was implemented to address the observed limitations. The architecture includes an input layer that accepts sequences of variable length with integer values representing tokens. An embedding layer is added, which maps each integer token to a dense vector representation of dimensionality `256`. The `mask_zero=True` parameter ensures that the layer ignores padding tokens, allowing the recurrent layers to skip these padding iterations during processing.

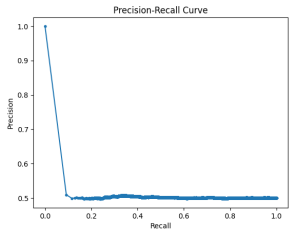
A Bidirectional LSTM layer with `32 units` is included to process the embedded sequences bidirectionally, capturing both past and future contexts. To mitigate overfitting, a dropout layer with a `0.5 dropout rate` is added. Finally, an output layer with a single neuron and `sigmoid activation function` is introduced for binary classification tasks. The model is compiled with the Adam optimizer and `binary crossentropy` loss function, with accuracy selected as the evaluation metric. Callbacks are defined for model checkpointing to save the best-performing model during training.

Metric	Value
Test Accuracy	0.862
Test Precision	0.876
Test Recall	0.843
Test F1-Score	0.859





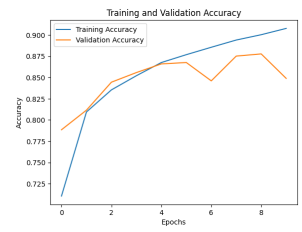


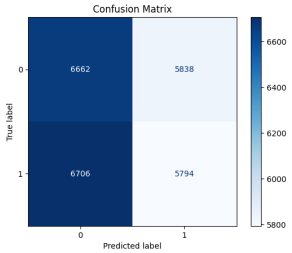


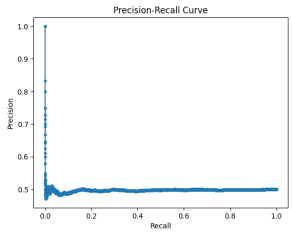
0.4 Model with GloVe Embeddings

In this section, pretrained GloVe embeddings are integrated into a sequence-to-sequence model to enhance representation. The embeddings are downloaded and processed to create a dictionary mapping words to vectors, and an embedding matrix is constructed for the dataset's vocabulary. Words not found in the embeddings are initialized with zeros.

The embedding layer uses the pretrained vectors, is set as non-trainable, and masks zero values to ignore padding tokens. The model architecture includes an input layer, the pretrained embedding layer, a Bidirectional LSTM layer for bidirectional context, a dropout layer (rate 0.5) to prevent overfitting, and a dense output layer with a sigmoid activation for binary classification while it is compiled with the Adam optimizer and binary crossentropy loss.







Metric	Value
Test Accuracy	0.834
Test Precision	0.834
Test Recall	0.833
Test F1-Score	0.834

0.5 Support Vector Machine (SVM)

The SVM-based text classification pipeline uses a CountVectorizer to turn text into numerical features and an SVM classifier with an RBF kernel to classify the data. The CountVectorizer captures single words and word pairs (using unigrams and bigrams) while keeping the vocabulary size limited to 5000 to make it more efficient. The SVM classifier is set up to balance accuracy and error handling using a regularization parameter (C=1) and automatically adjusts to the data with a kernel coefficient (gamma=scale). The pipeline performs well, achieving accuracy scores of 86.22% on the validation set and 86.35% on the test set. Metrics like precision, recall, and F1-score show it works consistently well for both positive and negative classes.

Metric	Value
Test Accuracy	0.863
Test Precision	0.863
Test Recall	0.863
Test F1-Score	0.863

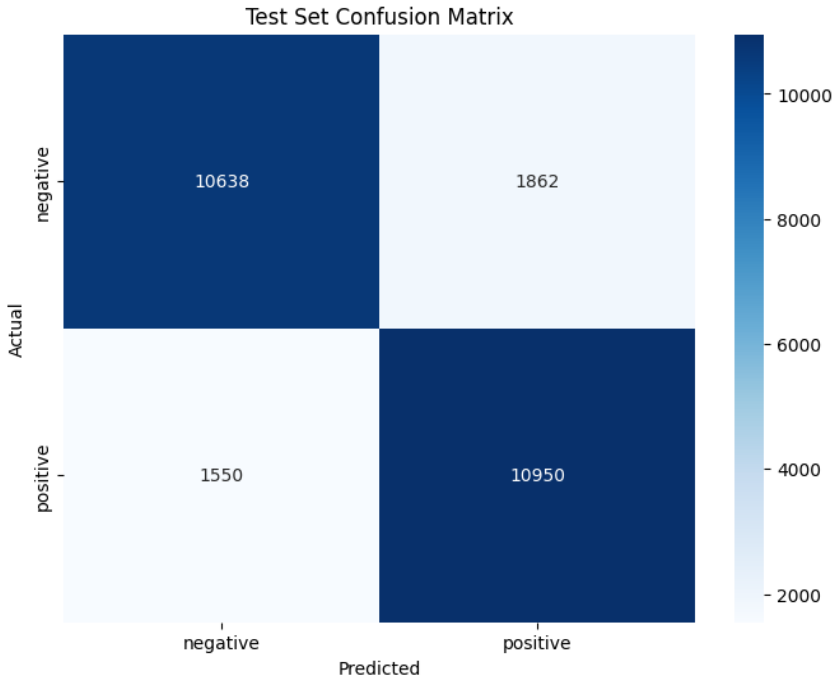


Figure 4: LSTM.

Comparative Metrics for All Methods

To compare the performance of all methods used in this project, the following table summarizes the metrics:

Method	Accuracy	Precision	Recall	F1-Score	Training Time
Support Vector Machine (SVM)	0.8635	0.8637	0.8635	0.8635	469.44
UniGram Model	0.884	0.876	0.895	0.885	401.18
BiGram Model	0.894	0.909	0.875	0.892	773.81
TriGram Model	0.895	0.894	0.896	0.895	1124.0
Sequence-to-Sequence LSTM Model	0.862	0.876	0.843	0.859	340.0
GloVe Embeddings Model	0.834	0.834	0.833	0.834	294.0

The model training process was conducted using an NVIDIA A100 GPU, available through Google Colab Premium.